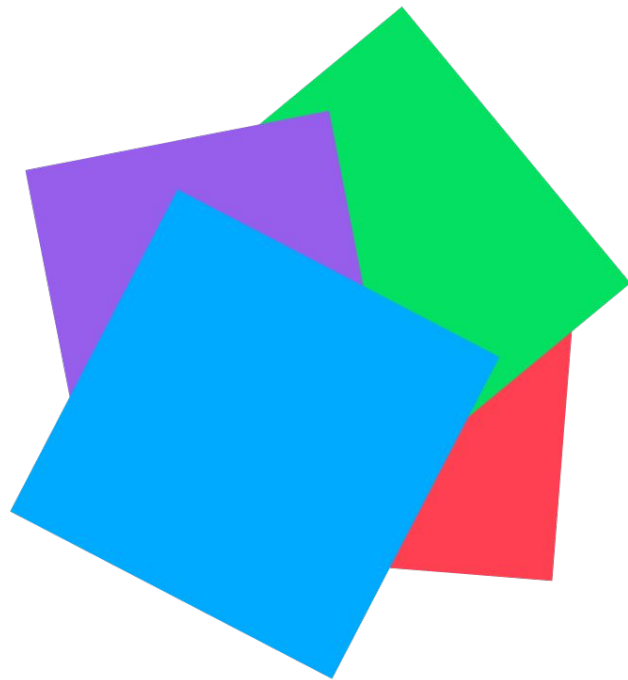


PostgreSQL и работа с данными



Сегодня разберём, как:



Создавать схему и управлять миграциями через goose



Выполнять запросы через pgx и управлять соединениями



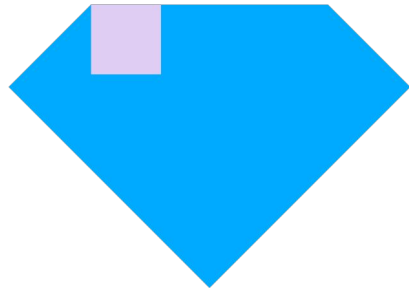
Читать планы запросов и подбирать индексы



Работать со snapshots, MVCC и транзакциями

После занятия вы сможете

Подключить PostgreSQL к приложению,
объяснить план запроса и выбрать индекс,
сохранить корректность при конкуренции.

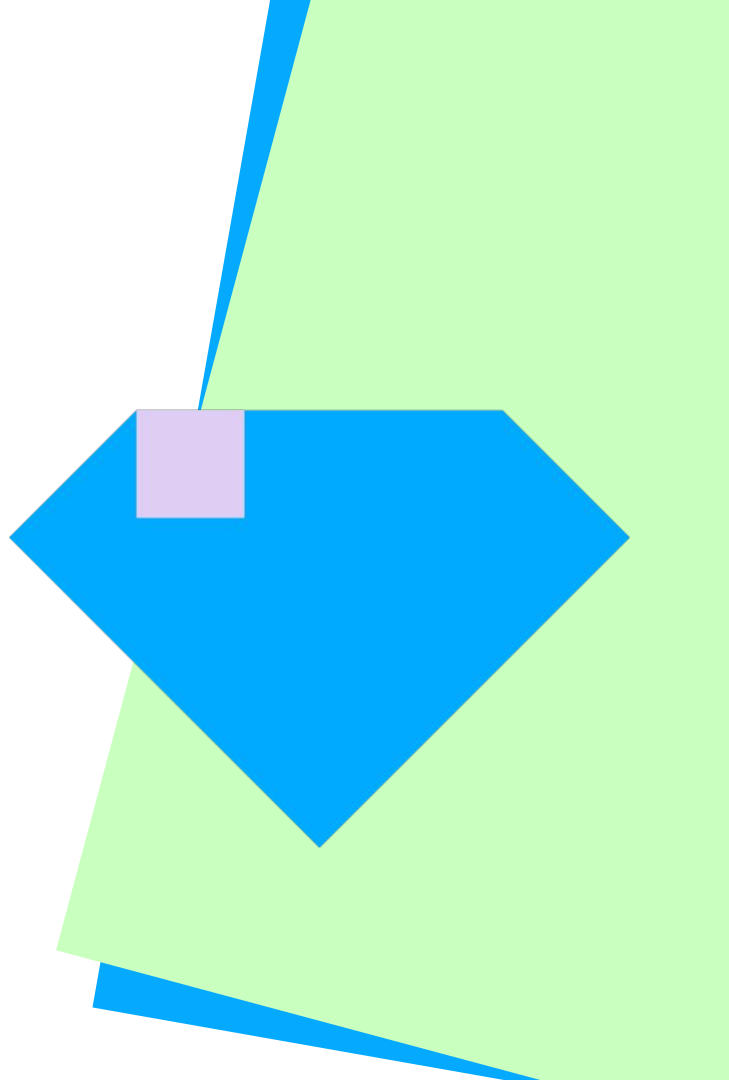


План занятия

- I. DDL/DML, миграции и goose
- II. Запросы из Go: pgx, пул, параметры
- III. Планы запросов и индексы
- IV. Транзакции: snapshot, MVCC, изоляция
- V. Сборка запросов: Squirrel
- VI. Пул соединений: PgBouncer

I. Миграции

- 01 DDL и DML:
схема и данные
- 02 Что такое миграции
и зачем они нужны
- 03 goose: последовательные
миграции, Up и Down



DDL и DML

В чём разница между DDL и DML?

Группа А

```
CREATE TABLE films (  
    id bigint,  
    title text  
);  
ALTER TABLE films  
    ADD COLUMN rating numeric;
```

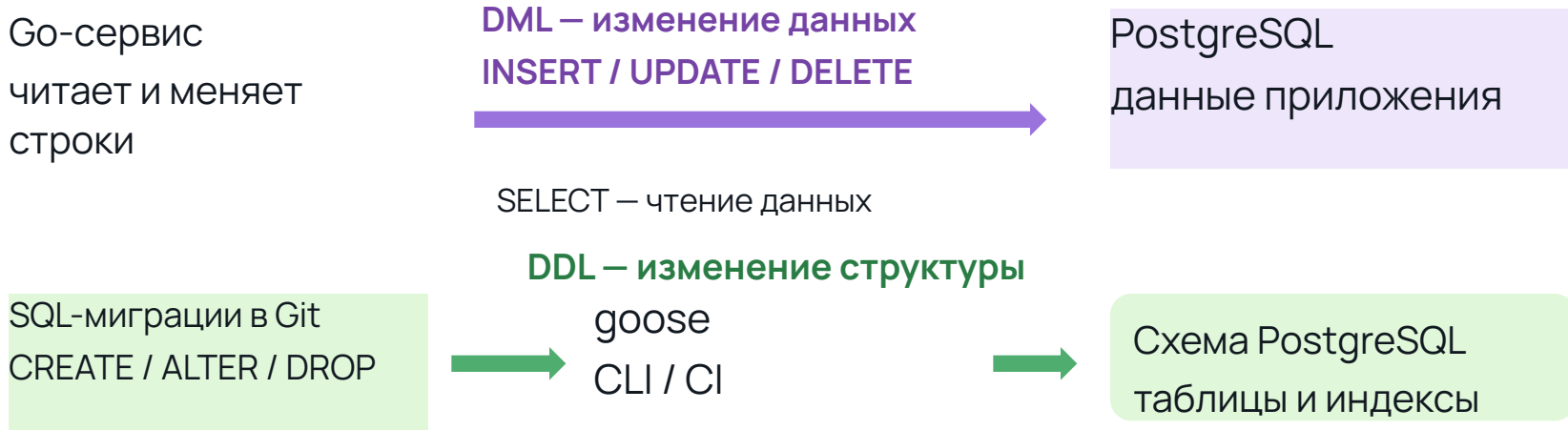
Группа Б

```
INSERT INTO films (id, title)  
VALUES (1, 'Arrival');  
  
UPDATE films SET rating = 9  
WHERE id = 1;  
  
DELETE FROM films WHERE id = 1;
```

Какая группа команд меняет структуру БД, а какая — строки таблицы?

Что такое миграции и зачем они нужны?

Миграции — versionированные изменения структуры и данных БД.



goose применяет миграции по порядку: 00001 → 00002 → 00003.

В примере миграции меняют схему; они также могут содержать DML.

Пакет goose: установка и первая миграция

Что делает goose

Хранит историю схемы.

Применяет SQL-файлы по порядку.

Работает из CLI и как Go-библиотека.

1 Установить CLI

```
go install \
github.com/pressly/goose/v3/cmd/goose@latest
```

2 Подготовить каталог и подключение

```
mkdir -p migrations
export GOOSE_DRIVER=postgres
export GOOSE_DBSTRING="postgres://user:pass@localhost/mydb"
```

3 Создать миграцию с номером 00001

```
goose -dir migrations -s create create_films sql
# migrations/00001_create_films.sql
```


SQL-миграция: Up и Down в одном файле

Один файл для применения и отката миграции.

```
-- 00001_create_films.sql
-- +goose Up
CREATE TABLE films (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  title TEXT NOT NULL,
  release_date DATE NOT NULL,
  rating NUMERIC(3,1) CHECK (rating BETWEEN 0 AND 10)
);
CREATE INDEX films_release_date_idx ON films(release_date);

-- +goose Down
DROP TABLE films;
```

По умолчанию — одна транзакция. CREATE INDEX CONCURRENTLY требует отдельной миграции с — +goose NO TRANSACTION.

Основные CLI-команды goose

GOOSE_DRIVER и GOOSE_DBSTRING уже заданы в окружении.

Статус миграций

```
goose -dir migrations status
```

Применить все новые миграции

```
goose -dir migrations up
```

Применить только одну

```
goose -dir migrations up-by-one
```

Откатить последнюю

```
goose -dir migrations down
```

Откатить до версии 1

```
goose -dir migrations down-to 1
```

Текущая версия

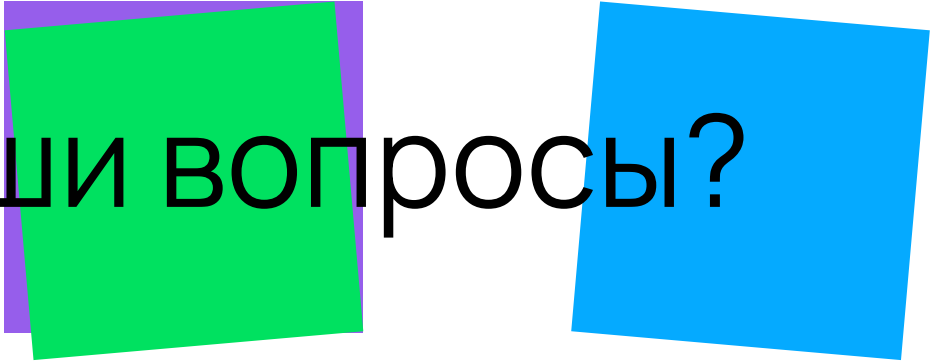
```
goose -dir migrations version
```

Создать миграцию 00002

```
goose -dir migrations -s \  
  create add_feature sql  
# 00002_add_feature.sql
```

Применить до версии 2

```
goose -dir migrations up-to 2
```



Ваши вопросы?

II. Подключаем приложение к БД

- 01 Пакеты database/sql, pgx
- 02 Создание соединения с БД. Пул соединений
- 03 Заккрытие соединения при graceful shutdown
- 04 Выполнение запросов
- 05 SQL-инъекции и параметры запросов

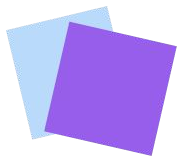
database/sql и драйвер PostgreSQL

database/sql — общий API и менеджер пула из стандартной библиотеки Go.

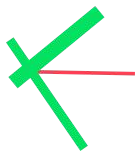
Драйвер — адаптер, который знает протокол СУБД.
Для PostgreSQL можно использовать pgx через stdlib.

Пакет pgx

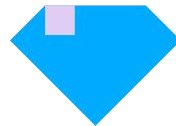
Драйвер PostgreSQL для Go



Высокая
производительность



Пул соединений:
pgxpool



Можно логировать все
запросы

[Официальный сайт](#)

pgx: одна строка и набор строк

QueryRow — одна строка; Query — набор строк.

Для команд без строк результата используем Exec.

```
var title string
err := pool.QueryRow(ctx,
    `SELECT title FROM films
    WHERE id = $1`, filmID,
).Scan(&title)
if errors.Is(err, pgx.ErrNoRows) {
    return ErrFilmNotFound
}
return err
```

```
rows, err := pool.Query(ctx,
    `SELECT id, title, release_date
    FROM films`)
if err != nil { return err }
defer rows.Close()
for rows.Next() {
    var film Film
    err = rows.Scan(&film.ID,
        &film.Title, &film.ReleaseDate)
    if err != nil { return err }
    // используем film
}
return rows.Err()
```

QueryRow сообщает ошибку в Scan. После Query закрываем rows и проверяем rows.Err(): ошибка возможна во время чтения.

Создание соединения с БД. Пул соединений

1 соединение выполняет
запросы по очереди
Новое соединение для
каждого запроса
требуется ресурсов

Используем пул соединений

- 1 Переиспользует открытые соединения
- 2 Позволяет выполнять запросы параллельно
- 3 Ограничивает число соединений к БД
- 4 Заменяет оборвавшиеся соединения

Создание пула соединений

```
cfg, err := pgxpool.ParseConfig(dsn)
if err != nil { return nil, err }
cfg.MaxConns = 20
cfg.MinConns = 2
cfg.MaxConnLifetime = time.Hour
cfg.MaxConnIdleTime = 30 * time.Minute
cfg.HealthCheckPeriod = time.Minute

pool, err := pgxpool.NewWithConfig(ctx, cfg)
if err != nil { return nil, err }
if err := pool.Ping(ctx); err != nil {
    pool.Close()
    return nil, err
}
return pool, nil
```

Как выбрать MaxConns

Пример бюджета:
100 соединений БД
– 20 резерва = 80.

4 процесса × 20 = 80.

Считайте максимум реплик,
обновления и фоновые задачи.

Уточняйте по нагрузке:
ожидание пула, задержки, CPU БД.
Это пример, не готовый рецепт.

Зачем нужен graceful shutdown

Резкая остановка приложения

Запросы обрываются: клиент может не получить результат.

Незавершённая транзакция откатывается при потере сессии.

COMMIT мог пройти до обрыва ответа. Результат нужно проверить.

Порядок завершения

1. Остановить новый трафик.
2. Дождаться текущих запросов в пределах таймаута.
3. Остановить фоновые задачи.
4. Закрыть пул соединений.

Graceful shutdown: сначала HTTP, затем БД

srv уже запущен; pool используется HTTP-обработчиками.

```
sigCtx, stop := signal.NotifyContext(
    context.Background(), os.Interrupt, syscall.SIGTERM)
defer stop()
<-sigCtx.Done()

shutdownCtx, cancel := context.WithTimeout(
    context.Background(), 10*time.Second)
defer cancel()
if err := srv.Shutdown(shutdownCtx); err != nil {
    log.Printf("shutdown: %v", err)
    if err := srv.Close(); err != nil {
        log.Printf("force close: %v", err)
    }
}
pool.Close()
```

Для Shutdown нужен новый context. Фоновые задачи завершаем до pool.Close().

SQL-инъекции

SQL-инъекция позволяет изменить смысл запроса к БД

Если приложение вставляет пользовательский ввод в SQL, злоумышленник может изменить запрос и выполнить свой код

Good luck speed cameras



SQL-инъекция: ввод меняет смысл запроса

Небезопасная сборка SQL

```
username := r.FormValue("username")
query := fmt.Sprintf(
    "SELECT id FROM users "+
    "WHERE username = '%s'",
    username,
)
err := pool.QueryRow(ctx, query).
    Scan(&id)
```

' закрывает строковый литерал.

OR TRUE делает условие истинным.

– комментирует остаток SQL.

Ввод пользователя: ' OR TRUE –

```
SELECT id FROM users
WHERE username = '' OR TRUE --'
```

Фильтр пропускает все строки и злоумышленник получает данные о всех пользователях

Как защититься от SQL-инъекций

SQL-шаблон и значения передаются отдельно.

Пользовательский ввод остаётся данными.

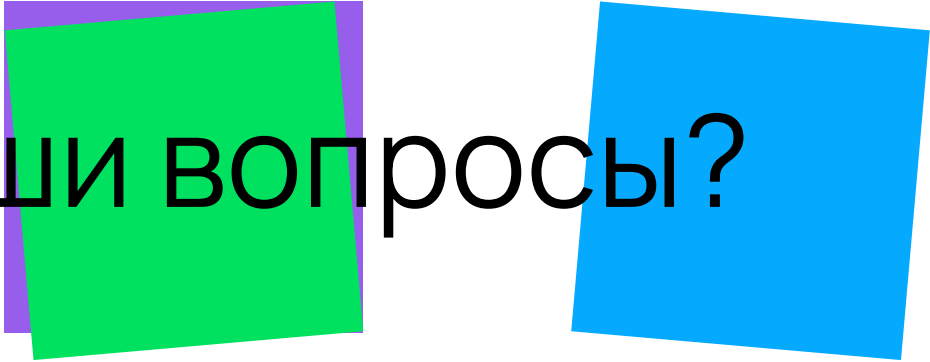
Параметры для значений

```
username := r.FormValue("username")
var id int64
err := pool.QueryRow(ctx,
    `SELECT id FROM users
    WHERE username = $1`,
    username,
).Scan(&id)
```

Структура SQL — из allowlist

Имена колонок и ASC / DESC
нельзя подставить через \$1.
Выбирайте их из списка в коде.

Валидация проверяет бизнес-правила;
права БД ограничивают ущерб.



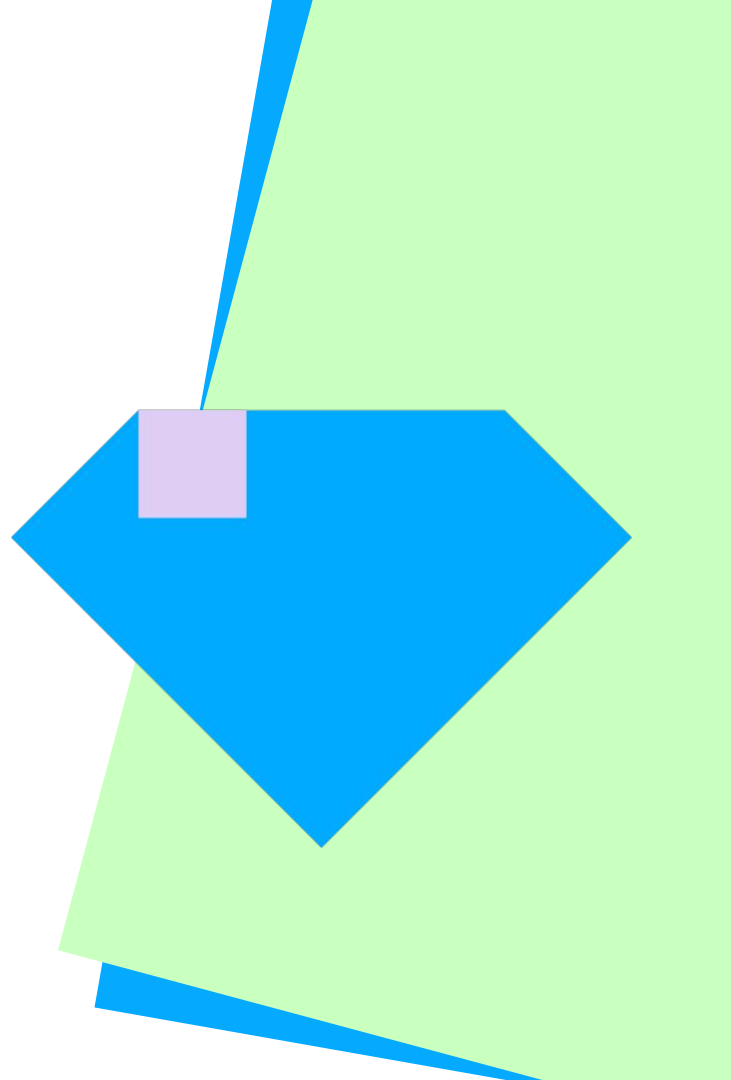
Ваши вопросы?

III. Планы запросов и индексы

01 Как получить и прочитать
план запроса

02 Устройство, типы
и составные индексы

03 Как индекс меняет план:
два примера до и после



Как PostgreSQL выбирает план

1. SQL и схема

Что нужно получить: фильтры, связи, сортировка и ограничения.

2. Варианты выполнения

Как читать и соединять данные: сканирования, join, sort.

3. Оценка стоимости

Статистика + размеры + настройки → оценка строк и затрат.

4. Исполнение

Executor выполняет выбранное дерево операций.

Из рассмотренных вариантов планировщик выбирает самый дешёвый по оценке.

Неверная оценка числа строк может привести к неудачному выбору.

Как посмотреть план запроса

EXPLAIN: оценка

```
EXPLAIN
SELECT title
FROM films
WHERE id = 424242;
```

```
-- запрос не выполняется
-- получаем оценки
```

EXPLAIN ANALYZE: выполнение

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT title
FROM films
WHERE id = 424242;
```

```
-- запрос выполняется
-- получаем фактические rows
```

ANALYZE действительно выполняет запрос, включая изменения данных.

На занятии используем SELECT; BUFFERS показывает обращения к страницам.

Читаем дерево плана: от источника к результату

```
Limit
-> Sort
    Sort Key: created_at DESC, id DESC
    -> Seq Scan on films
        Filter: (tenant_id = 42)

-- схематичная форма, без результатов замера
```

Seq Scan читает и фильтрует → Sort упорядочивает → Limit возвращает 20 строк.

Читаем связи снизу вверх; исполнитель запрашивает строки сверху вниз.

Что означают поля EXPLAIN

Поле	Что показывает	Как интерпретировать
cost=a..b	Оценка: старт / весь узел	Условные единицы; не миллисекунды
rows / width	Оценка строк / ширины	rows — выход узла, width — байты
actual time	Фактическое время	мс; среднее за один запуск узла
rows / loops	Факт: строки / запуски	$\text{rows} \times \text{loops} \approx$ всего выданных строк
Buffers	Обращения к страницам	hit — кеш PostgreSQL; read — загрузка
Filter / Cond	Отбор строк / поиск	Сравниваем оценки с фактом снизу вверх

Узлы плана: какой способ работы выбран

Узел	Что делает	На что обратить внимание
Seq Scan	Читает таблицу	Нормально для большой доли строк
Index Scan	Индекс → строки таблицы	Index Cond и число обращений к heap
Bitmap Scan	TID → страницы таблицы	Промежуточная селективность; Recheck
Index Only	Читает из индекса	Heap Fetches; карта видимости
Sort / Join	Сортирует / соединяет	Память, temp, строки и число loops

Зачем нужен индекс

Индекс — дополнительная структура доступа к данным.
Он помогает найти небольшую часть строк и иногда получить
нужный порядок без отдельной сортировки.

Что выигрываем

Меньше просмотренных страниц.
Быстрый поиск и диапазоны.
ORDER BY ... LIMIT без Sort.

За что платим

Место на диске и в кеше.
Работа при INSERT / UPDATE.
Создание и обслуживание.

Типы индексов PostgreSQL

Тип	Типичная задача	Устройство / особенность
B-tree	Равенство, диапазон, порядок	Сбалансированное дерево; по умолчанию
Hash	Только равенство	Хеш-бакеты; без порядка
GIN	Массивы, JSONB, full text	Элемент → набор ссылок на строки
GiST	Диапазоны, геометрия, KNN	Расширяемое дерево поиска
SP-GiST	Разбиение пространства	Например, trie / quad / k-d tree
BRIN	Большие упорядоченные данные	Сводки диапазонов страниц; recheck

Составной индекс: порядок ключей

```
CREATE INDEX films_tenant_recent_idx  
ON films (tenant_id, created_at DESC, id DESC);
```

```
SELECT id, title  
FROM films  
WHERE tenant_id = 42  
ORDER BY created_at DESC, id DESC  
LIMIT 20;
```


Почему индекс есть, а в плане — Seq Scan

Наличие индекса ещё не повод его использовать.

Если запрос возвращает большую долю таблицы, последовательное чтение может оказаться дешевле множества обращений через индекс.

Проверяем причину

Селективность и размер таблицы.

Статистику после ANALYZE.

Условие, типы, порядок ключей.

Повторяем измерение

Один запрос и одинаковые данные.

Оценки строк → факт → Buffers.

Учитываем прогрев кеша.

IV. Транзакции и MVCC

Границы и ACID → snapshot и версии строк

Изоляция → DBTX и TxManager → service layer

Как увидеть согласованные данные,
пока другие транзакции их меняют?

Граница транзакции

Перевод 100 ₽: списать со счёта 1 и зачислить на счёт 2.

Явная транзакция нужна, когда несколько действий должны зафиксироваться вместе или требуют общего снимка / блокировок.

Отдельные транзакции

Каждый запрос фиксируется отдельно.
Ошибка второго запроса не отменяет завершённое списание.

Одна бизнес-операция

BEGIN объединяет оба изменения.
COMMIT фиксирует их вместе.
При ошибке выполняем ROLLBACK.

Как выглядит транзакция

```
BEGIN;
```

```
UPDATE accounts  
SET balance = balance - 100  
WHERE id = 1;
```

```
UPDATE accounts  
SET balance = balance + 100  
WHERE id = 2;
```

Оба шага успешны

```
COMMIT;
```

Ошибка / отмена

```
ROLLBACK;
```

Все команды выполняются на одном соединении.

Завершение — COMMIT либо ROLLBACK; это альтернативные исходы.

ACID: что гарантирует транзакция

A — Atomicity

Списание и зачисление фиксируются вместе или оба отменяются.

C — Consistency

Перевод сохраняет общую сумму. Это обеспечивают код и ограничения.

I — Isolation

Чужой незавершённый перевод не виден. Другие гарантии зависят от изоляции.

D — Durability

Успешный COMMIT переживает сбой БД при обычных настройках надёжности.

Транзакция не придумывает бизнес-правила: инварианты нужно выразить в коде, ограничениях и правилах конкурентного доступа.

Snapshot: какую версию данных мы читаем

T1: баланс 100 → 80
COMMIT ещё не выполнен

T2: обычный SELECT
Какая версия видима?

100
видно T2

80
пока видно только T1

Snapshot задаёт, какие версии строк видимы читателю.

Это основа согласованного чтения при параллельных изменениях.

MVCC: несколько версий одной строки

До UPDATE

balance = 100
версия A

UPDATE

После UPDATE

balance = 100
старая версия

balance = 80
новая версия

- UPDATE создаёт новую версию строки.
- Snapshot определяет видимую версию.
- Обычный SELECT не блокирует UPDATE.
- VACUUM очищает уже ненужные версии.

Что такое изоляция?

Изоляция определяет, какие изменения параллельных транзакций мы можем увидеть и какие аномалии допускаем.

Слабее изоляция

- Больше допустимых аномалий.
- Для части правил нужны блокировки или проверки.
- Меньше гарантий от СУБД.

Строже изоляция

- Больше аномалий запрещено.
- Возможны ошибки сериализации.
- Нужно повторять транзакцию целиком.

Выбираем гарантии для бизнес-правил; производительность проверяем измерениями.

Уровни изоляции транзакций

SQL определяет четыре уровня; PostgreSQL реализует три варианта поведения.

Read Uncommitted

В PostgreSQL работает как Read Committed: грязных чтений нет.

Read Committed

Новый снимок для каждого SELECT. Повторное чтение может увидеть изменения.

Repeatable Read

Снимок первого запроса сохраняется до конца транзакции. Возможен конфликт.

Serializable

Гарантия последовательного результата. При конфликте повторяем всю транзакцию.

По умолчанию — Read Committed. В pgx уровень задаёт поле TxOptions.IsolationLevel.

Когда появляется и сколько живёт снимок

Read Committed

Новый снимок на каждую команду. Два SELECT могут увидеть разное.

Repeatable Read

Один снимок с первой команды, требующей снимка, до конца транзакции.

BEGIN и первый запрос

BEGIN; пауза; SELECT...

Снимок нужен при чтении, а не в момент BEGIN.

Свои изменения видны

Транзакция видит результаты своих предыдущих команд.

Snapshot задаёт видимые версии. От изоляции зависят срок действия снимка и другие гарантии; Serializable также отслеживает конфликты.

Read Committed: новый снимок для SELECT

T1: два чтения

```
BEGIN ISOLATION LEVEL READ COMMITTED;  
SELECT balance  
FROM accounts WHERE id = 1; -- 100  
  
-- После COMMIT в T2  
SELECT balance  
FROM accounts WHERE id = 1; -- 80  
COMMIT;
```

T2: изменение между чтениями

```
BEGIN;  
UPDATE accounts  
SET balance = 80  
WHERE id = 1;  
COMMIT;
```

Порядок: первое чтение T1 → COMMIT в T2 → второе чтение T1.
Новый снимок видит уже зафиксированный баланс 80.

Repeatable Read и Serializable

Repeatable Read

```
BEGIN ISOLATION LEVEL REPEATABLE READ;  
SELECT balance  
FROM accounts WHERE id = 1; -- 100  
-- T2 изменила баланс и сделала COMMIT  
SELECT balance  
FROM accounts WHERE id = 1; -- 100  
COMMIT;
```

Снимок первого запроса стабилен.
Свои изменения видны.

Serializable

```
BEGIN ISOLATION LEVEL SERIALIZABLE;  
SELECT balance  
FROM accounts WHERE id = 1;  
UPDATE accounts  
SET balance = balance - 10  
WHERE id = 1;  
COMMIT;  
-- Возможен конфликт: 40001
```

40001 возможен и до COMMIT.
Повторяем всю транзакцию.

Внешние эффекты требуют отдельного решения: outbox или идемпотентность.
Повтор транзакции не должен повторно отправлять письмо или списывать деньги вне БД.

Один интерфейс для pool и tx

```
type DBTX interface {  
    Exec(ctx context.Context, sql string, args ...any) (  
        pgconn.CommandTag, error)  
    Query(ctx context.Context, sql string, args ...any) (  
        pgx.Rows, error)  
    QueryRow(ctx context.Context, sql string, args ...any) pgx.Row  
}
```

Этот интерфейс реализуют `*pgxpool.Pool` и `pgx.Tx`.

Репозиторий принимает DBTX и выполняет запросы через него.

Интерфейс менеджера транзакций

```
type TxManager interface {  
    WithinTransaction(  
        ctx context.Context,  
        opts pgx.TxOptions,  
        fn func(context.Context, DBTX) error,  
    ) error  
}
```

Менеджер передаёт открытую транзакцию в callback.
Ошибка callback → ROLLBACK; успех → попытка COMMIT.

Реализация на pgx

```
func (m *Manager) WithinTransaction(
    ctx context.Context, opts pgx.TxOptions,
    fn func(context.Context, DBTX) error,
) error {
    tx, err := m.pool.BeginTx(ctx, opts)
    if err != nil { return err }
    defer func() {
        cleanup, cancel := context.WithTimeout(context.WithoutCancel(ctx), 3*time.Second)
        defer cancel()
        _ = tx.Rollback(cleanup)
    }()
    if err := fn(ctx, tx); err != nil { return err }
    return tx.Commit(ctx)
}
```

Отмена ctx не откатывает транзакцию: Rollback получает отдельный context.

Использование в сервисе

```
return s.txManager.WithinTransaction(  
    ctx,  
    pgx.TxOptions{IsoLevel: pgx.ReadCommitted},  
    func(ctx context.Context, q DBTX) error {  
        if err := s.balanceRepo.Withdraw(  
            ctx, q, from, amount,  
        ); err != nil {  
            return err  
        }  
        return s.balanceRepo.Deposit(ctx, q, to, amount)  
    },  
)
```

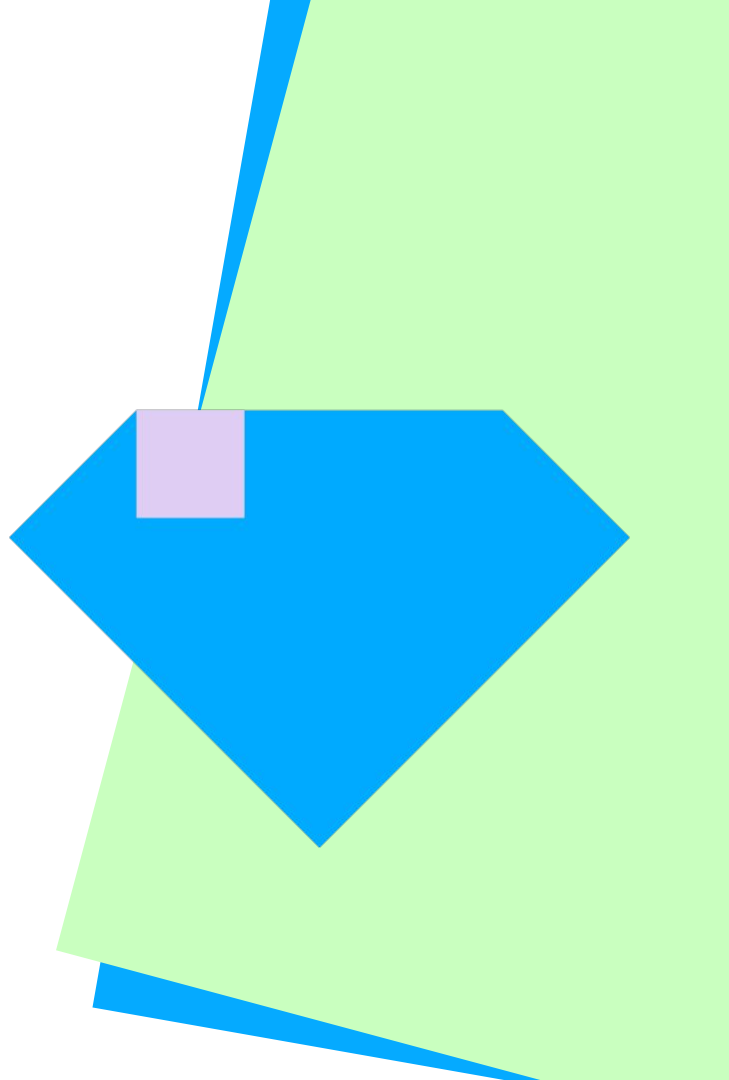
Сервис определяет бизнес-границу. Оба репозитория используют один q. Менеджер отвечает за BEGIN, COMMIT и ROLLBACK.

V. Пакет Squirrel

01 Зачем нужны сборщики запросов

02 Как использовать Squirrel

03 Примеры



Squirrel: собираем запрос из условий

Итоговый SQL

```
SELECT id, title
FROM films
WHERE rating >= $1
      AND title ILIKE $2
ORDER BY id
LIMIT 10;
```

```
-- args: 8.0, "%matrix%"
```

Тот же запрос через builder

```
q := sq.StatementBuilder.
    PlaceholderFormat(sq.Dollar).
    Select("id", "title").
    From("films").
    Where(sq.GtOrEq{"rating": 8.0}).
    Where("title ILIKE ?", "%matrix%").
    OrderBy("id").Limit(10)
```

```
sqlText, args, err := q.ToSql()
```

Каждый Where добавляет условие через AND. Значения остаются в args.

PlaceholderFormat(sq.Dollar) заменяет ? на \$1, \$2, ...

Squirrel: необязательные фильтры

Добавляем условие, только если пользователь указал фильтр.

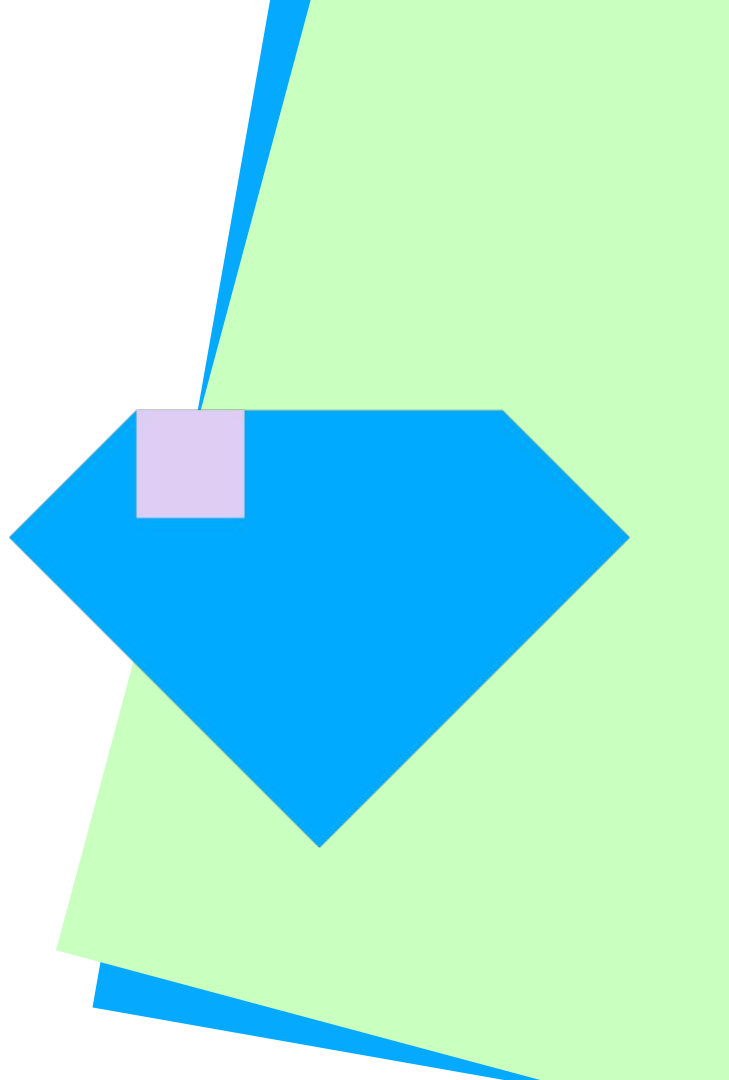
```
q := sq.StatementBuilder.PlaceholderFormat(sq.Dollar).  
    Select("id", "title").From("films")  
if minRating != nil {  
    q = q.Where(sq.GtOrEq{"rating": *minRating})  
}  
if title != "" {  
    q = q.Where("title ILIKE ?", "%" + title + "%")  
}  
sqlText, args, err := q.OrderBy("id").Limit(10).ToSql()  
if err != nil { return err }  
rows, err := pool.Query(ctx, sqlText, args...)
```

VI. PgBouncer

01 Зачем он нужен

02 PgBouncer и pool в приложении

03 Режимы работы



PgBouncer: зачем он нужен

20 экземпляров приложения
× 50 соединений = 1000
соединений к PostgreSQL.

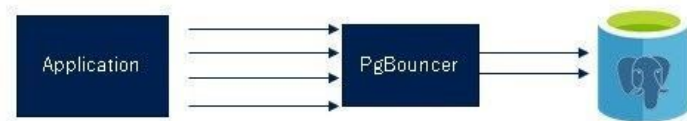
PgBouncer — отдельный пулер
перед базой данных.

Он позволяет многим клиентам
делить меньшее число
серверных соединений.

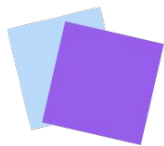
No Connection Pooling



Connection Pooling using PgBouncer proxy service

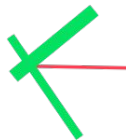


Пул в приложении и PgBouncer



pgxpool

Работает внутри одного экземпляра приложения. Переиспользует его соединения и ограничивает локальный параллелизм.



PgBouncer

Работает как отдельный сервис перед PostgreSQL. Обслуживает несколько экземпляров и ограничивает общее число соединений к БД.



Вместе

pgxpool управляет соединениями процесса. PgBouncer ограничивает общее число подключений к PostgreSQL. Если серверные соединения заняты, клиенты ждут в очереди.

Режимы пула PgBouncer

Режим	Освобождение	Особенности
session	После отключения клиента	Все возможности PostgreSQL Состояние сессии сохраняется
transaction	После завершения транзакции	Обычно для stateless-сервисов Состояние сессии не гарантируется
statement	После каждого SQL-выражения	Только autocommit Транзакции из нескольких выражений запрещены

Итоги занятия

- Миграции: воспроизводимая схема и проверенный откат.
- Пул и SQL: бюджет соединений, параметры вместо склейки.
- Планы: оценки строк, actual, Buffers и выбранные операции.
- Индексы: под условия и порядок; учитываем цену записи.
- Snapshot: согласованность чтения и срок жизни старых версий.
- Транзакции: бизнес-граница, изоляция и обработка ошибок.
- Squirrel и PgBouncer: сборка запросов и управление подключениями.

Проверяем результат и затраты на выполнение запроса.